

Ask What Changed, Not What to Order: Typed Shock Hypotheses, Compiled into an Inventory Controller, Activated by an Anytime-Valid e-process

[PENDING: author list]

[PENDING: affiliation]

[PENDING: email]

Abstract—A language model can be attached to a deployed replenishment controller in three separable ways: by changing *when* it is consulted, *what* it may say, and *when* its answer is trusted. Published agentic inventory systems change all three at once, so no reported gain can be attributed to any of them. We separate the three. COLLIE consults a frozen language model only at rare trigger moments, and the model may return only a typed hypothesis about the exogenous process, a SHOCKSPEC, drawn from a closed vocabulary of six shock families and frozen at the proposal period. A deterministic compiler maps that hypothesis onto one point of a 72 configuration grid for a capped base-stock controller, which computes every order, and the hypothesis influences orders only after an anytime-valid e-process started strictly after the proposal crosses the Ville threshold $1/\alpha_j$ under the geometric budget $\alpha_j = \alpha 2^{-j}$ over at most two proposals per episode. The resulting guarantee is a model-conditional, anytime-valid, per-episode false-activation bound, and we state explicitly what it does not cover. The testbed is an audited reimplement of InventoryBench whose accounting matches the official pipeline exactly and whose operations research baseline reproduces all 1,320 published order sequences bit for bit. That reproduction surfaced a harness divergence: the published baseline score of 0.4447 belongs to the legacy simulator, while the same policy scores 0.5208 under the authoritative submission path. We evaluate an eleven-rung arm ladder with physical and charged call accounting kept separate. [PENDING: headline result]

Index Terms—large language models, inventory control, operations research, anytime-valid inference, e-process, sequential change detection, agent evaluation

I. INTRODUCTION

Retailers are already asking language models what to order. On InventoryBench, a benchmark of 1,320 multi-period lost-sales replenishment instances, the strongest published language-model strategy reaches a normalized reward of 0.5380 against 0.4447 for the operations research policy it is asked to improve [1]. That gap is why the approach is taken seriously. It is also uninterpretable: the published strategy calls the model once per period, lets it name an order quantity outright, and applies that quantity with no test of whether the model was right, so three decisions are welded into one design and the reported gain cannot be assigned to any of them.

Reproducing the baseline made this worse. Our arm 1 reimplements the published capped base-stock policy and matches all 1,320 published order sequences bit for bit, so it is the same policy rather than an approximation of one. Under the

benchmark’s authoritative submission path that policy scores 0.5208, not 0.4447, because the published figure came from the benchmark’s legacy simulator, which keeps a lost shipment visible in in-transit inventory forever and so makes the baseline under-order after every loss. The motivating evidence is therefore both bundled and partly artifactual, and the useful question is not whether a language model helps on average but which of the three decisions does the work.

The thesis

A language model should not be asked what to order. It should be asked what changed, once, in a vocabulary the system fixed in advance, and it should be believed only after the data says so.

COLLIE implements that in three stages. A shared trigger decides *when*: an alert or a calibrated detector opens a proposal opportunity, a refractory window suppresses repeats, and at most two proposals are permitted per episode. A closed schema decides *what*: at the proposal period τ_j the model returns a SHOCKSPEC, a typed claim about the exogenous process drawn from a registered vocabulary of families, directions, onset windows, magnitude bins and durations, and it cannot write an order, a threshold, a likelihood, a falsifier, or code. An anytime-valid e-process decides *whether*: the hypothesis is frozen at τ_j and influences no order until evidence arriving strictly after τ_j drives the process above $1/\alpha_j$, and until then a deterministic compiler is the only thing that ever touches the baseline controller’s parameters. Our notes name the design axis in one line: compile shocks, or always inventory. Either the model’s reading of an exception is compiled once into the controller and then defended by prospective evidence, or the model is asked what to order, every period, forever.

One episode threads through the paper. A planner is told that a carrier has suspended outbound scans on a lane and that containers booked over the last week may not arrive at all. This is family 5 of our generator, a lost-shipment burst in which orders dispatched in one to three consecutive periods are never delivered, and it is a hard case on purpose: the demand stream is untouched, so the calibrated detector fires at its null rate and the shortfall shows up in telemetry only as an arrival that does not happen. Text is the only early signal, and a wrong reading of it is expensive in both directions.

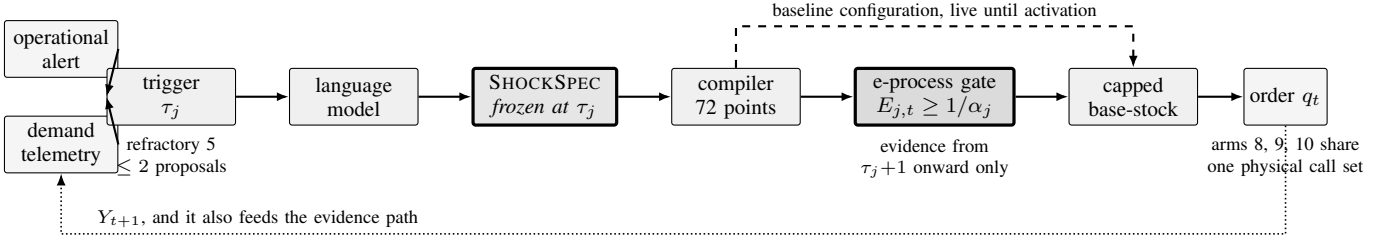


Fig. 1. The handshake. A shared trigger decides *when* the model is consulted, a closed schema decides *what* it may say, and an anytime-valid e-process decides *whether* the answer touches an order. The gate is drawn as a valve rather than an arrow because that is the point: the compiled configuration is computed immediately and flows only after evidence gathered strictly after τ_j drives $E_{j,t}$ above $1/\alpha_j$. The dashed path is the baseline configuration, which is live until then. The dotted return is what makes fact (F1) of Section VI load-bearing rather than decorative, since the controller’s own orders shape the observations the test consumes.

Contributions

- **An audited testbed, and a divergence finding.** Our runner matches the official pipeline with a maximum absolute difference of exactly 0.0 over 648 comparisons, and arm 1 reproduces every published baseline order sequence bit for bit, which is how we established that the published score belongs to the legacy simulator (Section II).
- **SHOCKSPEC and its compiler.** A closed, frozen, typed vocabulary for what changed, plus a total deterministic map from a hypothesis to one point of a 72 configuration grid over a capped base-stock controller (Sections IV, V).
- **Per-episode anytime-valid activation.** A discrete-mixture likelihood-ratio e-process started strictly after the proposal, with budget $\alpha_j = \alpha 2^{-j}$ over at most two proposals, a family-by-family validity statement, and one explicitly prohibited shortcut (Section VI).
- **An eleven-rung ladder with honest cost accounting,** in which arms 8, 9 and 10 share one physical set of proposal calls, so a difference between them cannot be a difference in calls (Section VIII).

This draft is written against a running experiment, so every result cell is marked **[PENDING:]** and inventoried, and Section VI-C states the single guarantee the system certifies and enumerates the readings of it that are wrong.

II. THE AUDITED BENCHMARK

What does InventoryBench specify, and what did we have to establish ourselves? We answer from its code and its data, because three answers differ from its documentation and one of them moves a published number.

Inventory and lead times. There are 1,320 instances, 720 synthetic and 600 derived from real weekly apparel sales, split evenly across three lead-time directories. Synthetic episodes run 50 decisions and real ones 47, not the 48 often reported, which counts the trajectory file’s header line. Two lead-time quantities are easy to conflate: the *promised* lead time, passed to the policy at initialization with values $\{0, 4, 2\}$, and the *actual* per-period lead time, never shown to the policy, with values $\{0\}$, $\{4\}$ or $\{1, 2, 3, \infty\}$, where ∞ encodes a lost shipment. Exactly 440 instances, the whole stochastic third, contain at least one lost period.

Event order and observability. Each period decides, then receives, then sells, then charges. Two consequences are load-bearing: an order with actual lead time zero arrives in the same period, because scheduling precedes the arrival pop, and the in-transit total shown at t still contains units that land at t . Holding falls on ending inventory and nothing is backordered, which makes lost-sales units a coherent safety endpoint. The policy sees uncensored previous demand rather than sales, and in-transit inventory is a scalar with no shipment ages, so many shipment-level histories are consistent with the same receipts. That last fact drives a decision in Section VI: the controller may keep an imputed first-in-first-out ledger, and the verifier may never use it as evidence.

Integer economics, and two absences. Every demand, profit and holding cell is an integer, so every sum is exact and our equivalence suite can demand a maximum absolute difference of exactly 0.0 rather than a tolerance: a nonzero difference could only be a semantic disagreement. Holding cost is 1.0 everywhere and profit is 19, 4 or 1, so **the labels high, med and low name the margin, not the holding burden**. Two absences are ours to fill. There is no upstream order cap, so COLLIE registers its own C_t and labels it as ours wherever it affects a number, distinct from the published baseline’s internal smoother. And a transit pause is not expressible in the instance format, because both simulators fix a shipment’s arrival period at order time, so it needs a project-owned sidecar.

A. The harness divergence

The benchmark ships two simulators and they disagree about one thing. In the legacy game simulator a lost order stays in in-transit inventory forever, since its arrival period is set to infinity and never popped, while in the submission path it is never added to in-transit at all. The disagreement never changes the physics, since replaying a run’s orders under the other simulator reproduces the arrivals, the reward and the lost-unit total exactly. It changes what a policy *reads*, because a base-stock policy under legacy semantics credits units that will never arrive, orders less, and loses fewer units.

We declare the submission path authoritative, because it is the contract we are scored against. Under legacy semantics our reimplementations reproduces the published 0.4447 to

one unit in the last place and matches all 1,320 published order sequences bit for bit, a far stronger check than a matching aggregate that many policies could reach. Under the authoritative path the same policy scores 0.5208, exactly the 880 deterministic-lead-time instances still match, and all 440 stochastic ones differ. Since 440 is precisely the number of instances the audit found to contain a lost shipment, the divergence is accounted for with no residue. The published figure was therefore generated under legacy lost-order semantics and is not reachable from the submission path, so every table comparing against it must name the harness.

III. PROBLEM SETUP

What must the information structure respect for a sequential test to be legitimate inside a closed control loop?

Let I_t be on-hand inventory at the start of period t , P_t the aggregate in-transit total, A_t our order, R_t aggregate receipts, and p and h the per-unit profit and holding cost. The order is coerced and capped as $A_t = \min(\max(0, \lceil q_t \rceil), C_t)$ and dispatched with a latent transit time, and our endpoints accumulate profit and lost sales under the audited event order. Let Y_t be the exposed observable, uncensored demand in the headline setting, and $\mathcal{F}_t$ the sigma-algebra generated by every observation through period t , taken *immediately before* A_t is chosen. Two properties follow. **(A1) Actions are predictable:** A_t is $\mathcal{F}_t$ -measurable, and the runner builds and freezes the observation before calling the controller, so this holds by construction rather than by convention. **(A2) Actions affect only the future:** A_t influences Y_{t+1} and later, never Y_t . Conditioning Y_r on $(\mathcal{F}_{r-1}, A_{r-1})$ is therefore well defined and does not condition on anything A_{r-1} could have altered retroactively, and every density below is written that way. A zero actual lead time moves R_t rather than Y_t , so (A2) survives. Finally, τ_j is the proposal period of hypothesis j and a stopping time, h_j the SHOCKSPEC frozen there, and $E_{j,t}$ its evidence process.

IV. SHOCKSPEC: THE TYPED COMMITMENT

What is the model allowed to say? Exactly one object, drawn from a vocabulary the system froze before any test episode ran. Closing that vocabulary is what makes the later test admissible rather than decorative.

At the proposal period τ_j the model receives only information in $\mathcal{F}_{\tau_j}$: the permitted product text, any alert visible then, the demand and arrival history, the inventory position, an aged pipeline summary and the costs. It must return valid JSON under a closed schema that forbids unknown keys and yields an immutable object. The model selects nine fields. `shock_family` is one of `no_change`, `demand_level`, `temporary_pulse`, `lead_time_shift`, `shipment_loss`, `transit_pause` or `compound`. `target_stream` ranges over demand, arrival, both and none, and `direction` over the six registered directions. `onset_window` is a registered offset pair or null, while `magnitude_bin`, `persistence` and `duration_bin` take their registered bins. `evidence_refs` points at

visible message spans or observation identifiers, and `prospective_signature` is schema-defined. Five fields are stamped by the system and never by the model: τ_j , the one-based proposal index, the model identifier, the decoding hash and the prompt hash. `no_change` is a first-class output rather than a failure code, and an accepted abstention is scored separately from a formatting failure because capability and reliability should not share a cell. `transit_pause` is a first-class family even though no generator family emits it alone, since it is the supply leg of family 6.

The model selects a key. It cannot write a falsifier, a likelihood, a threshold, an order equation, or code: the system maps the pair `(shock_family, prospective_signature)` to a verifier construction and the whole object to controller parameters. An invalid object gets one format-repair attempt, a second failure falls back to the unchanged baseline, and every attempt is counted as accepted, accepted after repair, or fallback. The restriction is load-bearing: if the model could invent its own vague test after seeing the trigger window, falsifiability would be cosmetic. Two structural facts remove the circularity. **(A3) Closed selection space:** the hypothesis is drawn from a registered vocabulary whose map to a construction is frozen. **(A4) Frozen at a stopping time:** τ_j is a stopping time, h_j is $\mathcal{F}_{\tau_j}$ -measurable, and its evidence product starts at $\tau_j + 1$. For the carrier suspension the model is expected to return `shipment_loss` on the arrival stream, direction `arrival_interrupted`, duration bin `1_3`, with `evidence_refs` naming the alert and the arrival residual. Nothing in that object is an order. It says what changed.

V. COMPILATION: FROM A CLAIM TO AN ORDER

How does a sentence about the world become an order quantity? Through a total, pure, deterministic function with no episode state, history or clock.

A `CONTROLCONFIG` is a four-tuple $(m, L_{\text{eff}}, \gamma, k)$ with $m \in \{0.6, 0.75, 1.0, 1.25, 1.5, 2.0\}$, $L_{\text{eff}} \in \{1, 2, 3, 4\}$ and $\gamma \in \{0, 0.5, 1\}$, which is $6 \times 4 \times 3 = 72$ configurations. The fourth entry is a pure function of the other three, obtained by comparing the point against the grid reference on two flags, whether the compiled belief moves demand and whether and how it moves the supply side. That key has two consumers: control acts on it, and Section VI's registry resolves the same key into the construction that will test it, so a hypothesis cannot be acted on under one reading and tested under another. Note that L_{eff} is a compiled *belief* and never the observation's promised lead time, because the compiler is a pure function of the SHOCKSPEC alone. Precedence is fixed as family, then direction, then magnitude bin, then target stream.

A lead-time shift moves L_{eff} alone, because arrivals are still coming, while a shipment loss moves γ alone, because arrivals keep their timing but fewer can be trusted to be real. A pause is deliberately asymmetric, since under the authoritative harness nothing inflates the pipeline count for γ to correct. Composition is reference-relative, because an absolute grid point applied wholesale would move axes the hypothesis does

TABLE I. THE REGISTERED COMPILER TABLE, BY SEVERITY RANK, AS ABSOLUTE POINTS AROUND THE GRID REFERENCE (1, 1, 1).

Shape	m	L_{eff}	γ
demand_level up	1.25 / 1.5 / 2.0	1	1.0
demand_level down	0.75 / 0.6 / 0.6	1	1.0
temporary_pulse up	1.25 / 1.5 / 2.0	1	1.0
temporary_pulse down	0.75 / 0.6 / 0.6	1	1.0
lead_time_shift	1.0	2 / 3 / 4	1.0
shipment_loss	1.0	1	0.5 / 0.0 / 0.0
transit_pause	1.0	2 / 2 / 3	1.0 / 1.0 / 0.5
compound	1.25 / 1.25 / 1.5	2 / 2 / 3	1.0
no_change	1.0	reference	1.0

not concern, and a demand-up hypothesis compiled to $L_{\text{eff}} = 1$ against a promised lead time of 2 cancels its own demand signal exactly. An axis left at the reference is inherited, and a moved axis applies absolutely for m , as an offset for L_{eff} and as a ratio for γ .

One controller serves every arm that routes through the compiler, differing only in which configuration is active in a period. With $\bar{d}_t$ and $s_{d,t}$ the running mean and sample standard deviation of the training samples followed by every demand observed so far, and $z^* = \Phi^{-1}(p/(p+h))$,

$$IP_t = I_t + \gamma P_t, \quad (1)$$

$$S_t = m(1 + L_{\text{eff}}) \bar{d}_t + z^* \sqrt{1 + L_{\text{eff}}} s_{d,t}, \quad (2)$$

$$q_t = \min(\max(0, \lceil S_t - IP_t \rceil), C_t, \hat{C}_t). \quad (3)$$

The critical fractile is read from the per-period cost columns rather than assumed constant, putting z^* at the 0.95, 0.80 and 0.50 quantiles across the three registered cost ratios, and $\hat{C}_t$ is the published baseline’s own internal smoother, kept separate from our cap C_t because they are different objects that share a word. The property that makes the ladder legitimate is that at the per-instance baseline configuration this controller reproduces arm 1 bit for bit, so “the published operations research policy is running” is literally true for every arm while no hypothesis is active. For the running example the compiled point is $\gamma = 0.5$ at low severity and $\gamma = 0$ above it, so the inventory position shrinks by the discounted pipeline and the controller replaces the shipments it no longer believes in. That is the whole intervention.

VI. VERIFIED ACTIVATION

When may a frozen hypothesis touch an order? Only when observations that arrived after it was written down have paid for the privilege.

A. Construction and theorem

The model chooses h_j because something looked unusual, and reusing the periods that motivated the choice as confirming evidence would make the apparent evidence an artifact of selection, and (A3) and (A4) remove that. For a proposal j with registered null p_0 and alternative region Θ_j ,

$$E_{j,t} = \int_{\Theta_j} \prod_{r=\tau_j+1}^t \frac{p_{h_j,\theta}(Y_r | \mathcal{F}_{r-1}, A_{r-1})}{p_0(Y_r | \mathcal{F}_{r-1}, A_{r-1})} dW_j(\theta), \quad (4)$$

with $E_{j,\tau_j} = 1$ by the empty-product convention. Two facts make (4) an e-process started at τ_j [2], [3]. **(F1) Unit conditional mean:** for fixed θ the ratio has conditional expectation one under the null, because the expectation integrates it against p_0 and leaves $\int p_{h,\theta} d\mu = 1$, which needs only a common dominating measure and $p_0 > 0$ wherever $p_{h,\theta} > 0$, and we check that shared support rather than assume it. **(F2) Mixing preserves the property:** the fixed- θ product is a nonnegative martingale with unit initial value, a test martingale, and a mixture over θ with prior W_j is again one by Fubini, which handles a composite alternative without estimating θ from the data being tested.

(F1) is where the closed loop is handled, because it holds *whatever policy generated* A_{r-1} , including one that reacts to $E_{j,t}$ itself, so an adaptive, self-referential controller does not invalidate the test. That is why the design insists actions be predictable. One implementation detail is easy to get wrong: with W_j discrete on a registered grid, the mixture sits *outside* the product, so each component carries its own cumulative ratio and the prior averages component wealth. Multiplying period-wise mixture averages would define a different process.

Ville’s inequality bounds a nonnegative supermartingale M with $M_0 = 1$ by $\mathbb{P}(\sup_t M_t \geq 1/\alpha) \leq \alpha$ [4]. Index proposals within episode e by $j = 1, 2, \dots$, allocate $\alpha_j = \alpha 2^{-j}$, and activate proposal j at the first t with $E_{j,t} \geq 1/\alpha_j$.

Theorem 1. *Under the registered global no-change null, with (A1) through (A4) and (F1), (F2),*
 $\mathbb{P}_0(\text{any false activation in episode } e) \leq \sum_j \alpha_j \leq \alpha \sum_{j \geq 1} 2^{-j} = \alpha.$

Each $E_{j,\cdot}$ is a unit-initial nonnegative martingale, so Ville bounds its own crossing probability by α_j , and a union bound with $\sum_{j \geq 1} 2^{-j} = 1$ gives the result. The allocation is why the proposal index is one-based: an index of zero would spend the whole budget on the first proposal.

B. Arrivals, and the one prohibited shortcut

Only the aggregate receipt is observed, and many shipment-level histories are consistent with the same receipt sequence, so *assuming* first-in-first-out order would inject an assumption as if it were data. The controller’s imputed ledger is a control convenience only and the verifier marginalizes instead, which is enforced rather than documented: the ledger’s symbols are unreachable from the verifier by a static check and a runtime input-graph assertion. Our own dispatch quantities are known exactly, so the only latent quantity is each outstanding order’s transit realization, and the state is the vector of remaining transit times of outstanding cohorts, truncated at L_{max} with a lost outcome absorbing, giving at most 1296 states for $L_{\text{max}} = 4$. A forward recursion over that state returns the *exact* conditional density of the receipt, so (F1) applies verbatim, with exactness checked per family against brute force.

Compound shocks need one more rule, because the generator’s conditional-independence flag is false there. Where the contract makes independence explicit a registered *joint* conditional likelihood may be used. Otherwise we run two

separate e-processes with an explicit prior split of the budget, and Theorem 1 applies with the finer partition, because the union bound is indifferent to how the budget is subdivided in advance. Multiplying marginal likelihood ratios when independence is not part of the contract is **prohibited**: under dependence the product is not the ratio of joint densities, its conditional mean need not be one, (F1) fails, and the bound is void with no visible symptom. The joint code path is therefore asserted unreachable whenever the flag is false, since an assertion is the only defense that survives a refactor.

C. Validity, lifecycle, and what we do not claim

A registered validity status accompanies every family and the code is asserted to agree with it, so a disagreement is a test failure. The two demand families use the mixture ratio and are anytime-valid, with the pulse underpowered by construction. The three arrival families and the compound family use the forward recursion, with a budget split for the compound case, and are anytime-valid *on our own episodes only*, since the official lead-time law’s probabilities are unpublished and a null with unknown parameters is not a registered null. The censored observation model and the plug-in variant are empirical only, the first pending derivation and the second permanently, because re-estimating p_0 from the stream being tested makes the null data-dependent and (F1) fails. The plug-in variant is implemented anyway, labeled as carrying no finite-sample guarantee.

A hypothesis is proposed on arrival and influences nothing, becomes *active* only on an activation point from the e-process, and is *refuted*, *expired* or *superseded* by registered operational rules. Only the transition into *active* is covered by the theorem, so exactly one guarantee is certified: arm 10’s activation rule is model-conditional, anytime-valid and per-episode. *Model-conditional* means it presumes the registered p_0 is the true conditional law of the exposed observable, so it is a statement within a model and not a distribution-free one. *Anytime-valid* means it holds at all stopping times simultaneously, so continuous monitoring and optional stopping are permitted. *Per-episode* means the union bound runs over proposals within one episode.

Nothing else is guaranteed. It is not dataset-wide family-wise control. It is not a safety guarantee, because activation being rare under the null says nothing about loss magnitude when activation does occur. It is not a profit guarantee, because an activated correct hypothesis can still lose money through a bad control mapping. It is not a bound on wrong-family activation: activating `transit_pause` when a `shipment_loss` is genuinely present is not a null event, so we report that as an empirical metric. It says nothing about refutation or expiry, and it does not extend to arrival-side families on official instances, to the censored observation model, or to the plug-in variant. We claim priority on no ingredient either: not the first event-triggered language model, not the first falsifiable commitment, not the first language-model-to-operations-research interface, and not a certified inventory agent.

VII. TRIGGERS AND THE ALERT BANK

Who decides when the model is consulted, and where does the text come from? Both are infrastructure rather than contribution, and we say so, because event-triggered invocation is already well covered [5]. What matters is that the infrastructure be shared exactly enough that no comparison can be confounded by it.

A proposal opportunity opens when an operational alert arrives or when a calibrated telemetry detector crosses its threshold. Six rules exist: alert only, CUSUM [6], Page-Hinkley, their disjunction with the alert channel, which is the primary rule, and two budget-matched schedules. Detectors are stateless about suppression: one emits a raw candidate and two wrappers decide, composed as `MaxProposals(Refractory( $d$ , 5), 2)`, so the window and the cap are uniform across rules, including rules written after the design was frozen. Thresholds come from Monte Carlo on development and calibration nulls only, at the nearest-rank 95th percentile of the per-episode maximum statistic over 60 stationary twin paths, so three of 60 null episodes fire by construction. The window of 5 periods covers the longest registered pulse and pause, and the cap of 2 is the value $\alpha_j = \alpha 2^{-j}$ was designed around. Two properties matter for the claims. The demand-side detectors read only the previous demand and the alert channel, both exogenous to the policy, so a trigger trace is a pure function of the episode’s demand and alert streams and two arms sharing a rule must produce bitwise-identical traces, which the suite asserts. Without that, a comparison between two arms would conflate a different invocation pattern with a different policy. And a denylist, enforced at runtime on every feature mapping and statically by a syntax-tree scan, forbids future demand and arrivals, pattern names, instance identifiers, split and test labels and every field of the hidden incident record. It is deliberately over-broad on substrings, because a trigger that peeks is worse than a bad trigger: it looks like a good one.

What the detector cannot see. Measured over 120 seeds per family at the frozen calibration under the no-alert condition, the firing rate is 0.99 for a demand increase, 1.00 for a demand decrease, 0.69 for the temporary pulse, 0.03 for the lead-time shift, 0.03 for the lost-shipment burst and 0.77 for the compound event. The two supply-only families sit at the null rate *by construction*, since a demand-side detector cannot see a lead-time shift or a lost shipment. This is not a tuning failure to be fixed: it is what makes the alert channel the only early signal for our running example.

The alert bank. The text channel is 180 short synthetic operational templates, 60 per split and 10 per family within each split, of which four per family-split cell are accurate, two overstated, two ambiguous and two operational distractors, with wording families held out across splits and controlled slots instantiated per episode. Template identity stays on the analysis side, because language-level uncertainty must treat the template as a sampling unit and not the rendered string, and synthetic messages are preferred because timing and

information content are controllable: a message must not carry information that could not have existed when it is delivered. Two independent raters score every template on five criteria, and leakage is not negotiable by consensus, since one below-threshold score from either rater removes it. **[PENDING: inter-rater agreement and templates pruned for leakage]** Each message also carries a hidden ALERTSPEC, and feeding that straight to the compiler is the perfect-parsing upper bound. Every trajectory is crossed with the identical exogenous realization under four information conditions, and the four cells of one seed are *paired replays of one independent unit*.

VIII. THE ARM LADDER AND HONEST ACCOUNTING

What cheaper thing would have worked just as well? Every rung in Table II is one answer, stated as a reviewer would state it, and the rung above it is what remains once the answer is removed. The same table carries the headline result slots, so the objection and the number that answers it sit on one line.

Arms 3, 5 and 11 reproduce the benchmark’s own three language-model strategies on a prompt builder asserted byte-identical to the published agent’s prompt at the same state, which is what makes arm 3 a fair representation of the existing approach rather than a strawman. Arm 7’s expiry equals the frozen refractory window, so it measures persistence rather than a longer invocation budget, and arms 4 and 6 are matched to arms 3 and 5 on *realized* calls. Arm 9’s guard is real: an observation contradicts the spec when it lands more than one baseline standard deviation on the wrong side of the pre-proposal baseline mean for the spec’s direction, and two consecutive contradictions refute it. Four further arms route through the identical compiler and controller, so a difference between any two is a difference in what reached it: a detector, a keyword parser, the hidden ALERTSPEC as the perfect-parsing ceiling, and an oracle SHOCKSPEC as the headroom bound. Only the last two may read hidden truth.

Arms 8, 9 and 10 build the same proposal prompt at the same decision point and differ only in the injected activation policy, and the cache collapses those to one physical request with one charged copy per arm. Two tests carry the design: one asserts that the physical call set is shared, the other asserts *divergence*, that identical proposals produce different activation traces. If they did not, the activation stage would be doing nothing.

Physical against charged. Every cost claim depends on this. A *physical* entry is what the provider saw, one per real request, and a *charged* entry is what an arm would have paid, one per arm per call it would have made, including calls served from cache. Arms 8, 9 and 10 sharing one proposal call therefore produce one physical and three charged entries, and $\text{physical} \leq \sum_{\text{arms}} \text{charged}$ becomes a property of the books, asserted over random arm sets. Every attempted call is classified exactly once when the arm settles it, an unsettled charged entry fails a conservation check, and dated cost is computed at log time from the registered price table. The configuration is cloud-only, with a hosted Gemini Flash endpoint as the primary at \$0.75 per million input and \$3.75

per million output tokens including thinking tokens, retrieved 2026-09-06, and a different provider for confirmation.

IX. EXPERIMENTAL DESIGN

What would count as evidence, and what was decided before any test trajectory ran? This section is design only. The experiments are running, so every cell is marked **[PENDING:]** and every table is a pre-built slot.

Splits, and the independent unit. Seeds come from disjoint reserved pools of 36 development units, 24 calibration units and 150 test units, six families by six, four and twenty-five seeds. Each test unit is crossed with the four information conditions, giving 600 paired replays plus 20 silent-null and 20 false-alert-null controls, so 640 held-out rollouts per sparse arm, with every threshold and mapping frozen on development and calibration data. **The independent unit is the seed, not the rollout.** The four information cells of one seed are paired replays of one exogenous trajectory, so treating them as four samples would inflate the effective sample size fourfold and narrow every interval by roughly half, invisibly. The evaluation engine is therefore registered to refuse any confirmatory statistic computed from a frame with more than one row per pair of independent unit and arm, and to raise rather than warn. Family tail estimates stay exploratory even at twenty-five seeds, and every language-level claim bootstraps jointly over seed and template identity.

Endpoints. The primary reward endpoint is cumulative undiscounted profit per episode and the primary safety endpoint is total lost-sales units per episode, both macro-averaged with equal weight across the six family strata and one null stratum and across the four conditions within a family. Fixing endpoint and weighting in advance prevents a later choice among raw profit, a ratio and a favorable mixture.

The three contrasts. Three contrasts and two endpoints give six tests, Holm-corrected as one family, and Table II holds their slots. C1 is arm 10 against arm 8: does verification earn its place, or is immediate activation enough? C2 is arm 10 against arm 6: does the typed hypothesis beat ephemeral numerical parameters at matched calls? C3 is arm 10 against arm 11: does it beat the strongest published method? C1 and C2 are mechanism contrasts and hold the trigger time, the information, the controller, the model, the decoding configuration and the realized call budget fixed, so the treatment is a single factor. C3 is not, and we do not present it as one, because arm 11 differs in timing, content and trust at once. Arm 2 is the registered secondary comparator for whether language is needed at all. Primary inference is two-sided paired randomization on per-seed differences after condition cells are aggregated within their seed, with seed-level cluster-bootstrap intervals and Wilcoxon tests as a secondary check, and every analysis not listed here is exploratory.

Three further reporting rules. False activation and wrong-family activation occupy separate columns and are never merged, because the first is covered by Theorem 1 and the second has no theorem-backed bound. Forty main-table null rollouts cannot estimate a five percent error rate, so a separate

TABLE II. THE LADDER AND THE HEADLINE RESULT SLOT IN ONE PLACE. EACH RUNG NAMES THE OBJECTION IT ANSWERS, AND ITS THREE CELLS ARE THE PRE-BUILT SLOTS THE SWEEP WILL FILL: STRATUM-WEIGHTED ESTIMATES ON THE SUBMISSION HARNESS WITH SEED-LEVEL CLUSTER-BOOTSTRAP INTERVALS OVER THE INDEPENDENT UNIT. THE LAST THREE ROWS ARE THE SIX CONFIRMATORY TESTS, HOLM-CORRECTED AS ONE FAMILY.

# Arm	Objection it answers	Profit	Lost sales	Norm. reward
1 capped base-stock	the published baseline	PENDING	PENDING	PENDING
2 telemetry-only adaptive	a changepoint detector would do this	PENDING	PENDING	PENDING
3 every-period direct action	just let the model order	PENDING	PENDING	PENDING
4 triggered direct action	the gain is from calling less often	PENDING	PENDING	PENDING
5 every-period parameters	the gain is from the OR layer	PENDING	PENDING	PENDING
6 sparse ephemeral parameters	the gain is from sparsity	PENDING	PENDING	PENDING
7 persistent parameters, fixed expiry	the gain is from persistence	PENDING	PENDING	PENDING
8 SHOCKSPEC, immediate	the gain is from typing the hypothesis	PENDING	PENDING	PENDING
9 SHOCKSPEC, heuristic rollback	a heuristic guard would do	PENDING	PENDING	PENDING
10 SHOCKSPEC, e-process	the contribution	PENDING	PENDING	PENDING
11 every-period recommendation, then model	the strongest published method	PENDING	PENDING	PENDING
CUSUM to compiler	does telemetry alone suffice	PENDING	PENDING	PENDING
keyword parser	does reading the message need a model	PENDING	PENDING	PENDING
ALERTSPEC upper bound	ceiling for perfect parsing	PENDING	PENDING	PENDING
oracle headroom	ceiling the compiler can reach at all	PENDING	PENDING	PENDING
C1, arm 10 vs arm 8	does verification earn its place	PENDING	PENDING	
C2, arm 10 vs arm 6	does typing beat ephemeral parameters	PENDING	PENDING	
C3, arm 10 vs arm 11	does it beat the best published method	PENDING	PENDING	

bank of 1,300 episodes runs the complete pipeline: 1,000 well-specified registered-model nulls across five strata, and 300 that violate those assumptions, reported separately and *never* pooled with them. And because an earlier call opportunity must not be mistaken for evidence that a message meant something, every semantic-content comparison fixes the alert timestamp, the trigger trace hash and the call opportunity across five variants, while the early-warning comparison is a *different estimand*. Efficiency reporting covers calls, repairs, rejections, tokens, latency quantiles, dated dollar cost and actions per accepted call, with profit-against-cost frontiers on *realized* budgets. One always-online sweep is $720 \times 50 + 600 \times 47 = 64,200$ calls, roughly \$108 to \$217 at the dated primary price. **[PENDING: activation ablation, null-audit calibration and fragility, content against timing, efficiency and frontiers, operational and hypothesis metrics, traced cases]**

X. RELATED WORK

Which ingredient is new? None. The handshake is.

Language models for operations. InventoryBench established the complementarity this paper starts from, and its four strategies are our arms 1, 3, 5 and 11 [1]. A design-time session can synthesize reusable operations research algorithms outright [7], and InvEvolve evolves white-box inventory policies online [8]. Our model does the opposite of both: it estimates the environment rather than the policy, writes no code, and every action stays fixed-form operations research. InvEvolve has no public code artifact as of our search, so any comparison against it must be labeled a faithful reimplementaion, and we have not implemented one. Field evidence supports the interface shape: judgmental adjustment of a forecast input raised profitability by 4.92 percent on average in a retail experiment [9], and average United States import shipping delay rose by 21 days between 2018 and 2024 [10].

When to call, and what to commit to. Event-triggered invocation with threshold, CUSUM, sequential probability ratio, Bayesian and learned rules is already established [5], so we treat invocation as fixed infrastructure and claim nothing about it. Structuring a plan as falsifiable commitments with confirming and falsifying evidence, and running a hypothesis lifecycle with provenance and belief updates, also exist already [11], [12]. What differs here is that the commitment parameterizes an *exogenous stochastic model* whose influence is authorized by a preregistered sequential statistic and compiled into inventory-control inputs, so the object is tested rather than merely recorded.

Statistics and inventory theory. We use e-values and anytime-valid inference as given [2], [13], with the classical sequential and changepoint machinery behind them [4], [6]. The nearest construction is the e-detector, which sums processes started at every time and controls the average run length [3], while we start one only at model-selected stopping times and control per-episode false activation by budget splitting. A theorem-backed *retirement* rule would need the forward and backward confidence-sequence intersection test [14], which we do not adopt, which is why our retirement rules are labeled empirical. The controller is the textbook one [15], and our transit-pause semantics follow an established simulation convention in which material already moving stops progressing [16].

XI. LIMITATIONS

What could make this paper wrong? Section VI-C bounds the theorem. Several of the real risks lie outside it.

The headroom band is not yet interpretable. The oracle-to-baseline headroom band we have measured so far was produced with a stand-in compiler and a scripted transport. It is narrow, and it is uninterpretable until the real pipeline lands end to end. That is the honest statement and we will not sharpen it. If the band stays narrow once the real compiler

and a real model are both in the loop, no arm between the baseline and the oracle can be distinguished and the study needs reframing. **[PENDING: headroom band under the integrated pipeline]**

Two families receive almost no proposals without an alert. Families 4 and 5 do not touch the demand stream, so the detector fires at its null rate by construction, every triggered arm degenerates to the baseline controller in those cells, and pooling them dilutes any effect the alert channel produces elsewhere. An arrival-side detector on expected against realized arrivals would be legal, and it is an open scope decision rather than something we have quietly done.

Reproducibility, open weights, and the oracle. The primary hosted endpoint rejects the decoding seed parameter and a second accepts it without honoring it, so reproducibility comes from a cache of stored bytes and a reader who re-derives our numbers from the provider will not get identical text. The project is cloud-only by a registered, dated operator decision, so the paper cannot make an open-weight claim. The oracle is an upper bound rather than a method: it reads the true family and onset, is never tuned on test data, and refuses the compound family, so compound rows are excluded and we state that wherever such a row appears. Verification is also late by design, which is why proposal-to-activation delay is a primary outcome. And the scope is controlled policy evaluation rather than deployment, since the real trajectories are genuine sales histories but their disruptions are simulated and true demand stays visible during a stockout. Redistribution is governed by upstream competition terms, so an artifact release carries code, generators, prompts, splits, manifests and hashes and directs users upstream for the data.

The registration is not yet frozen, and it gates the confirmatory claims. The preregistration file that must carry the endpoints, the stratum weights, the three contrasts, the Holm family, the budget allocation, the compiler mapping and the numeric go and kill thresholds does not yet exist. Until it does the contrast set of Section IX is provisional, no analysis can legitimately be labeled confirmatory, and the per-method freeze manifest is likewise unimplemented. The verifier and the schema parser are implemented on feature branches and are not yet integrated on the main line, and no pilot has been run, so no go or kill verdict exists. We say this plainly because the alternative is a paper whose method section describes a system its own evidence base does not yet contain. **[PENDING: integration status, pilot verdict, frozen registration hash]**

XII. CONCLUSION

Should a language model be asked what to order? The question is badly posed, because it silently bundles three separable decisions, and separating them is what makes any resulting gain attributable. COLLIE consults the model rarely, restricts it to a typed claim about what changed, compiles that claim into one point of a registered grid over a controller the operations research literature already trusts, and lets the claim touch an order only after an anytime-valid e-process started

strictly after the claim was frozen has paid for it. The model says what changed, once. Statistics decides whether to trust it. Operations research decides what to do.

Two things we built are useful whether or not the method wins. One is an audited testbed whose baseline reproduces every published order sequence bit for bit, which is what let us establish that the published baseline score belongs to one of the benchmark’s two simulators and is unreachable from the other, so any future comparison here has to name its harness. The other is a ledger that keeps physical requests separate from charged copies, which makes a shared proposal call across three activation policies a controlled ablation rather than a cost accident. **[PENDING: final result paragraph: which contrasts survived Holm correction, what the frontier showed, and which kill triggers fired]**

REFERENCES

- [1] J. Baek, Y. Fu, W. Ma, and T. Peng, “AI Agents for Inventory Control: Human-LLM-OR Complementarity,” *arXiv preprint arXiv:2602.12631*, 2026.
- [2] A. Ramdas, P. Grünwald, V. Vovk, and G. Shafer, “Game-Theoretic Statistics and Safe Anytime-Valid Inference,” *Statistical Science*, vol. 38, no. 4, pp. 576–601, 2023.
- [3] J. Shin, A. Ramdas, and A. Rinaldo, “E-detectors: A nonparametric framework for sequential change detection,” *The New England Journal of Statistics in Data Science*, vol. 2, no. 2, pp. 229–260, 2024.
- [4] J. Ville, *Étude critique de la notion de collectif*, ser. Monographies des Probabilités, Fascicule III. Paris: Gauthier-Villars, 1939.
- [5] Z. Wang, “Uncertainty-aware sequential decision rules for Event-Triggered LLM invocation in streaming systems,” in *Machine Learning and Knowledge Discovery in Databases (ECML PKDD)*. Springer, 2026, arXiv:2607.13048.
- [6] E. S. Page, “Continuous inspection schemes,” *Biometrika*, vol. 41, no. 1-2, pp. 100–115, 1954.
- [7] J. Baek, “LLMs Can Design Near-Optimal OR Algorithms,” *arXiv preprint arXiv:2608.27296*, 2026.
- [8] C. Huang, J. Lin, Z. Tang, B. Jiang, R. Jiang, B. Wang, and L. Wei, “InvEvolve: Evolving White-Box Inventory Policies via Large Language Models with Performance Guarantees,” *arXiv preprint arXiv:2605.00369*, 2026.
- [9] S. Kesavan, T. Kushwaha, and D. Steele, “Profit implications of judgmental adjustments to forecast inputs: Evidence from a Large-Scale field experiment,” *Management Science*, vol. 72, no. 1, pp. 119–127, 2026.
- [10] M.-J. Carreras-Valle and A. Ferrari, “The cost of delivery delays,” *AEA Papers and Proceedings*, vol. 115, pp. 618–623, 2025.
- [11] G. Liu, H. Zhao, and Q. Yao, “Falsifiable commitment planning for self-correcting web agents,” *arXiv preprint arXiv:2607.24167*, 2026.
- [12] I. Takahara and T. Mizoguchi, “Toward auditable AI scientists: A hypothesis evolution protocol for LLM agents,” *arXiv preprint arXiv:2607.09195*, 2026.
- [13] V. Vovk and R. Wang, “E-values: Calibration, combination and applications,” *The Annals of Statistics*, vol. 49, no. 3, pp. 1736–1754, 2021.
- [14] S. Shekhar and A. Ramdas, “Sequential Changepoint Detection via Backward Confidence Sequences,” in *Proceedings of the 40th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 202. PMLR, 2023, pp. 30 908–30 930.
- [15] A. J. Clark and H. Scarf, “Optimal policies for a Multi-Echelon inventory problem,” *Management Science*, vol. 6, no. 4, pp. 475–490, 1960.
- [16] L. V. Snyder, “Stockpyl: A Python package for inventory optimization and simulation,” in *Tutorials in Operations Research*. INFORMS, 2023, pp. 156–197.